

Data Scientist : les fondamentaux

Jour 1 / 3 — Introduction au Big Data et à la modélisation

Formateur : Mouaad AGOURRAM

Objectifs de la formation

- **Objectif opérationnel**

- Savoir appréhender le métier de Data Scientist.

- **Objectifs pédagogiques**

- Découvrir le métier de Data Scientist et les grandes familles de problèmes
- Savoir modéliser un problème de Data Science
- Créer vos premières modèles de machine learning
- Constituer votre boîte à outils de Data Scientist
- Participer à une première compétition

Prérequis : bases en programmation/scripting ; des bases de statistiques sont un plus.

Déroulé de la journée 1

Programme de la journée

1

Big Data

Écosystème & vocabulaire



2

Modélisation

Poser un problème de ML

1

Algorithmes

Régression linéaire / logistique



2

Outils

Python, Pandas, Scikit-learn

1

TP Titanic

Premier modèle

01

Introduction au Big Data

- Qu'est-ce-que le Big Data ?
- L'écosystème technologique du Big Data
- Introduction à la Data Science, le métier de Data Scientist

Qu'est-ce-que le Big Data ?

Définition : le Big Data désigne des volumes de données si importants, rapides ou variés qu'ils dépassent les capacités des outils classiques de gestion de bases de données, et nécessitent des architectures et méthodes spécifiques pour être collectés, stockés et exploités.



Volume

Quantité de données générées
(du Go à l'exaooctet)



Vélocité

Vitesse de génération et de
traitement (temps réel, flux)



Variété

Diversité des formats : structuré,
semi-structuré, non structuré



Véracité

Fiabilité, qualité et incertitude
des données

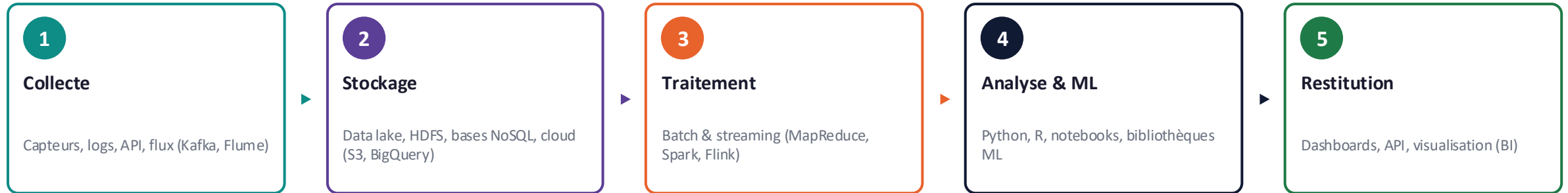


Valeur

Capacité à en extraire un
bénéfice métier concret

La multiplication des sources (capteurs IoT, web, transactions, réseaux sociaux...) et la diversité de leurs formes ont fait émerger un nouveau métier : le Data Scientist, chargé de rendre ces données exploitables pour la direction générale.

L'écosystème technologique du Big Data



Ce que ce paysage change pour le Data Scientist

- Les données ne tiennent plus forcément en mémoire sur un seul poste : besoin d'échantillonnage, de calcul distribué ou de traitement par lots.
- Les formats sont hétérogènes (JSON, logs, images, texte) : le nettoyage et la structuration deviennent une part majeure du travail.
- L'écosystème technique (data engineers, data lake, orchestration) fournit la matière première ; le Data Scientist la transforme en modèles exploitables.

Introduction à la Data Science, le métier de Data Scientist

Rôle du Data Scientist

- Rendre les données exploitables et leur donner du sens
- Traiter la donnée pour permettre à la direction générale d'adapter la stratégie de l'entreprise
- Un acteur-clé aux compétences multiples : statistiques, informatique, métier

Le triangle de compétences

- Mathématiques / Statistiques : modéliser, quantifier l'incertitude
- Informatique / Programmation : manipuler et industrialiser à l'échelle
- Connaissance métier : poser les bonnes questions, interpréter les résultats

Data Scientist

Professionnel qui conçoit, entraîne et évalue des modèles statistiques/ML à partir de données pour répondre à un problème métier.

Data Analyst

Se concentre sur l'exploration, la restitution et la visualisation de données existantes (moins de modélisation prédictive).

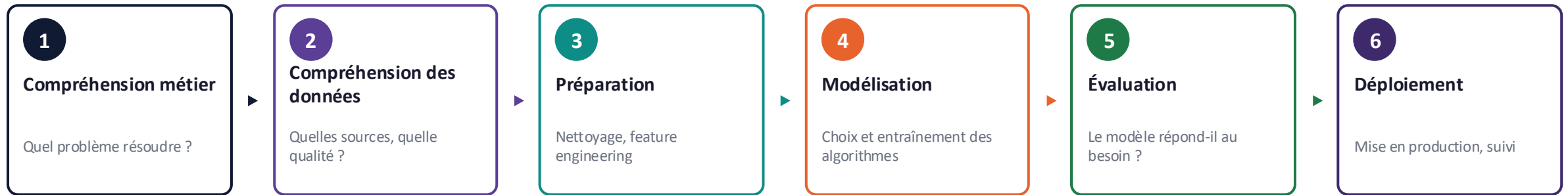
Data Engineer

Construit et fiabilise les pipelines qui collectent, stockent et mettent à disposition la donnée.

ML Engineer

Industrialise les modèles du Data Scientist : mise en production, API, suivi des performances (MLOps).

Le cycle de vie d'un projet Data Science (CRISP-DM)



CRISP-DM : un cycle, pas une ligne droite

- CRISP-DM (Cross Industry Standard Process for Data Mining) est le cadre méthodologique de référence en Data Science
- Les retours en arrière sont fréquents : une mauvaise performance en évaluation peut renvoyer vers la préparation des données, voire vers la compréhension métier
- Ce cycle structure les trois jours de cette formation : chaque TP reproduit une partie de ce processus, du cadrage jusqu'à l'évaluation en compétition

02

Modéliser un problème de Data Science

- Le vocabulaire d'un problème de Data Science
- De l'analyse statistique au machine learning
- Overview des possibilités du machine learning
- Modélisation d'un problème / Input-output

Le vocabulaire d'un problème de Data Science

Dataset	Ensemble des données disponibles pour le problème (lignes = observations, colonnes = variables).
Feature (variable)	Une colonne du dataset utilisée en entrée du modèle (aussi dite variable explicative ou prédicteur).
Target (cible / label)	La variable que l'on cherche à prédire (variable expliquée).
Training set	Sous-ensemble des données utilisé pour entraîner (ajuster) le modèle.
Test set	Sous-ensemble mis de côté pour évaluer objectivement la performance du modèle.
Modèle	Fonction mathématique apprise qui relie les features à la target.

Autres notions clés du vocabulaire

Paramètre	Valeur interne apprise par le modèle pendant l'entraînement (ex. les θ d'une régression).
Hyperparamètre	Réglage choisi avant l'entraînement, non appris par le modèle (ex. taux d'apprentissage, profondeur d'un arbre).
Epoch / itération	Un passage complet sur l'ensemble du training set pendant l'entraînement.
Pipeline	Enchaînement structuré des étapes de préparation et de modélisation, applicable de façon identique en entraînement et en prédiction.
Baseline	Premier modèle simple servant de référence pour mesurer l'apport de chaque amélioration ultérieure.
Fuite de données (data leakage)	Information de la target qui s'infiltré involontairement dans les features, faussant l'évaluation du modèle.

De l'analyse statistique au machine learning

Analyse statistique classique

- **Objectif** : expliquer et décrire (inférence)
- **Question type** : « pourquoi ? cet effet est-il réel ? »
- **Hypothèses fortes** : linéarité, normalité des erreurs...
- **Variables** : peu nombreuses, interprétabilité prioritaire
- **Résultat** : coefficients, p-values, intervalles de confiance
- **Validation** : sur les données d'ajustement (tests, résidus)
- **Exemples** : test d'hypothèse, régression linéaire classique
- **Titanic** : « le sexe a-t-il influencé la survie ? »

Machine Learning

- **Objectif** : prédire, généraliser à de nouvelles données
- **Question type** : « que va-t-il se passer pour ce cas ? »
- **Peu d'hypothèses a priori** : l'algorithme apprend des données
- **Variables** : très nombreuses, performance prioritaire
- **Résultat** : prédictions et score (accuracy, RMSE...)
- **Validation** : sur des données jamais vues (test set)
- **Exemples** : arbres, forêts aléatoires, réseaux de neurones
- **Titanic** : « quelle probabilité de survie pour ce passager ? »

C'est l'usage qui fait la différence, pas l'algorithme : la même régression logistique peut servir à comprendre ou à prédire. Première question à poser au métier lors du cadrage : « voulez-vous comprendre, ou prédire ? »

Overview des possibilités du machine learning



Apprentissage supervisé

Le modèle apprend à partir de données étiquetées (features + target connue) : classification, régression.



Apprentissage non supervisé

Aucune target : le modèle découvre des structures (clusters, dimensions) dans les données.



Apprentissage par renforcement

Un agent apprend par essais/erreurs en interagissant avec un environnement, guidé par une récompense.



Deep Learning

Réseaux de neurones profonds, particulièrement performants sur images, texte et signaux.

La suite de ce module se concentre sur l'apprentissage supervisé (classification / régression), point d'entrée classique du métier.

Exemples d'usage par famille d'apprentissage

Classification supervisée	Détection de fraude, diagnostic médical, filtrage de spam, prédiction de survie (Titanic)
Régression supervisée	Prévision de prix immobilier, prévision de la demande, estimation d'un délai de livraison
Non supervisé — clustering	Segmentation client, regroupement de documents similaires
Non supervisé — réduction de dimension	Visualisation de données à haute dimension, compression avant un modèle supervisé
Par renforcement	Optimisation de trajets logistiques, systèmes de recommandation dynamiques, jeux

Le choix de la famille dépend avant tout de la question posée et de la disponibilité (ou non) d'une target étiquetée — c'est la première décision de tout projet.

Modélisation d'un problème



Étapes clés de la modélisation

- Traduire une question métier (« qui va partir ? », « combien coûtera... ? ») en problème mathématique
- Choisir la nature de la target : catégorielle (classification) ou continue (régression)
- Identifier les features disponibles et pertinentes
- Choisir une famille de modèles adaptée, puis un critère d'évaluation

Découper les données : training set, validation set, test set



Training set	Sert à ajuster les paramètres du modèle (entraînement proprement dit).
Validation set	Sert à comparer plusieurs modèles / hyperparamètres sans « regarder » le test set — souvent remplacé par la cross-validation.
Test set	Réservé jusqu'à la toute fin : donne une estimation non biaisée de la performance finale du modèle.

Règle d'or : le test set n'est jamais utilisé pour choisir un modèle ou régler un hyperparamètre, sous peine de biaiser l'estimation de sa performance réelle.

Input / output d'un problème de machine learning

INPUT

- Matrice de features X (n observations $\times$ p variables)
- Éventuellement un vecteur target y (cas supervisé)
- Hyperparamètres de l'algorithme choisi



OUTPUT

- Une prédiction $\hat{y}$ pour chaque nouvelle observation
- Classification : une classe (ex. survécu / non) ou une probabilité
- Régression : une valeur numérique continue (ex. un prix)

TP OCR — Comment modéliser le problème de la reconnaissance optique de caractère ?

TP Mise en situation

La reconnaissance optique de caractères (OCR) consiste à identifier automatiquement des caractères à partir d'images. C'est un problème classique pour s'entraîner à formuler un problème de Data Science.

1. Formuler la question métier : « quel caractère est représenté sur cette image ? »
2. Identifier la nature du problème : classification multi-classes (une classe par caractère possible)
3. Définir l'input : chaque image est transformée en un vecteur de pixels (features)
4. Définir l'output : la classe prédite parmi l'ensemble des caractères possibles (ex. 0-9, A-Z)
5. Discuter en groupe : quelles difficultés pratiques (bruit, rotation, polices) et quelles features seraient utiles ?

03

Les familles d'algorithmes de machine learning

- Analyse supervisée
- Analyse non supervisée
- Classification / régression

Apprentissage supervisé

Définition — l'algorithme apprend une fonction reliant les features X à une target y connue sur les données d'entraînement, afin de généraliser cette fonction à de nouvelles observations.

Deux cas de figure, selon la nature de la target :



Classification

La target est une catégorie discrète : survécu/non, spam/non-spam, chat/chien/oiseau...

Question : « à quelle classe appartient cette observation ? »

Exemples : prédire la survie d'un passager, filtrer un spam, poser un diagnostic



Régression

La target est une valeur numérique continue : un prix, une température, une durée...

Question : « quelle valeur pour cette observation ? »

Exemples : estimer un prix immobilier, prévoir une demande, un délai de livraison

Les mêmes algorithmes servent le plus souvent aux deux cas : arbres de décision, forêts aléatoires, SVM, k-NN existent en version classification et en version régression. Seuls quelques-uns sont spécifiques : la régression logistique classe, la régression linéaire estime une valeur.

Chaque observation d'entraînement est donc une paire (features, target) : c'est ce couplage qui permet à l'algorithme de « superviser » son apprentissage.

Apprentissage non supervisée

Définition — aucune target n'est fournie : l'algorithme explore la structure interne des données (regroupements, dimensions dominantes) sans supervision externe.



Clustering

Regrouper les observations similaires (ex. segmentation client) — k-means, clustering hiérarchique.



Réduction de dimension

Résumer un grand nombre de variables en quelques composantes (ex. PCA) pour visualiser ou simplifier.



Détection d'anomalies

Identifier les observations atypiques par rapport au comportement majoritaire (fraude, panne...).

Classification / régression

	Classification	Régression
Nature de la target	Catégorie discrète (classe)	Valeur numérique continue
Question type	« Quelle classe ? »	« Combien ? »
Exemple	Le passager a-t-il survécu ? (oui/non)	Quel est le prix de ce bien ?
Sortie du modèle	Classe ou probabilité d'appartenance	Valeur numérique estimée
Métriques usuelles	Accuracy, precision, recall, ROC	MAE, RMSE, MAPE
Exemples d'algorithmes	Régression logistique, arbres, SVM	Régression linéaire, arbres de régression

04

Sous le capot des algorithmes

- La régression linéaire : fonction hypothèse, coût, descente de gradient
- La régression logistique : frontière de décision, coût convexe

Quelques rappels : fonction hypothèse, fonction convexe, optimisation

Fonction hypothèse

Modèle proposé pour relier X à y

$$h_{\theta} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_p x_p$$

une combinaison linéaire des features.

Fonction convexe

Fonction en forme de « cuvette » : un seul minimum global, ce qui garantit qu'un algorithme d'optimisation peut le trouver de façon fiable.

Optimisation

Processus qui ajuste les paramètres θ pour minimiser l'écart entre prédictions et valeurs réelles (la fonction de coût).

Ce qu'il faut retenir

- Entraîner un modèle = trouver les paramètres θ qui minimisent une fonction de coût
- La convexité de cette fonction assure que la descente de gradient converge vers le minimum global
- Plus il y a de features, plus l'espace des paramètres est grand — d'où l'importance du choix des variables

La construction de la fonction de coût

Fonction de coût (erreur quadratique moyenne)

$$J(\theta) = \frac{1}{2m} \sum (h_{\theta}(x_i) - y_i)^2$$

m : nombre d'observations
h θ (x_i) : prédiction du modèle
y_i : valeur réelle observée

Objectif : trouver ϑ qui minimise $J(\vartheta)$.

Pourquoi élever au carré ?

- Rend toutes les erreurs positives : sur- et sous-estimation comptent de la même façon
- Pénalise davantage les grandes erreurs que les petites
- Rend la fonction dérivable et convexe : condition nécessaire pour la descente de gradient

La dérivée de la fonction de coût et la mise à jour des paramètres

Gradient de la fonction de coût (par paramètre)

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

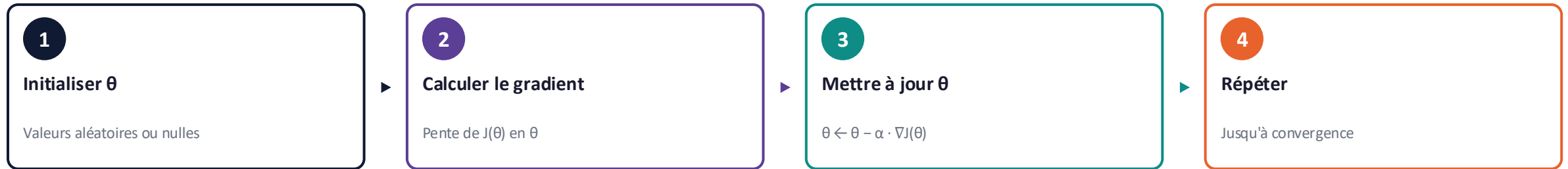
$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

Cette même logique de dérivation s'applique à la régression logistique, avec une expression du gradient à la forme quasi identique.

Comment lire cette formule

- Le gradient mesure, pour chaque paramètre θ_j , l'impact d'une petite variation sur l'erreur totale
- Le terme $(h_{\theta}(x_i) - y_i)$ est l'erreur de prédiction sur l'observation i
- On multiplie cette erreur par $x_{i,j}$: une feature avec des valeurs plus grandes a un impact plus fort sur la mise à jour
- Chaque paramètre est mis à jour simultanément, dans la direction opposée au gradient (la descente)

Méthode de minimisation : la descente de gradient



Pseudo-code

```
for iteration in range(n_iter):  
    grad = compute_gradient(theta, X, y)  
    theta = theta - alpha * grad
```

Le rôle du taux d'apprentissage α

- α trop petit : convergence très lente
- α trop grand : risque d'oscillation ou de divergence
- α bien choisi : convergence régulière vers le minimum de la cuvette convexe

Batch, stochastique, mini-batch : les variantes de la descente de gradient

Variante	Principe	Avantage	Limite
Batch (complet)	Utilise l'ensemble du training set à chaque mise à jour	Convergence stable et précise	Lent et coûteux en mémoire sur de gros volumes
Stochastique (SGD)	Utilise une seule observation à chaque mise à jour	Rapide, s'adapte au streaming de données	Trajectoire bruitée, convergence moins stable
Mini-batch	Utilise un petit sous-ensemble (ex. 32, 64, 128 observations)	Bon compromis vitesse / stabilité, exploite le calcul vectorisé	Nécessite de régler la taille du batch

Dans la pratique, scikit-learn utilise par défaut une résolution analytique ou une variante batch/quasi-Newton pour la régression linéaire simple ; SGD et mini-batch deviennent indispensables sur de très grands volumes ou en deep learning.

Sous le capot des algorithmes : la régression logistique

Régression logistique

Adapte la régression linéaire à la classification binaire en passant la combinaison linéaire des features dans une fonction sigmoïde, qui produit une probabilité entre 0 et 1.

Frontière de décision

Ligne (ou surface) qui sépare l'espace des features en régions associées à des classes différentes ; un point est classé selon le côté où il se trouve.

Fonction sigmoïde

$$\text{sigmoid}(z) = \frac{1}{1 + e^{-z}}$$
$$P(y = 1 | x) = \text{sigmoid}(\theta^T x)$$

Frontière de décision

- Dans un espace à 2 features, c'est une droite ; en dimension supérieure, un hyperplan
- Sa forme dépend de la complexité du modèle (linéaire ici, plus complexe pour d'autres algorithmes)

Règle de décision

- Si $P(y=1|x) \geq 0,5 \rightarrow$ classe 1
- Si $P(y=1|x) < 0,5 \rightarrow$ classe 0
- Le seuil 0,5 peut être ajusté selon le coût métier des erreurs (faux positifs vs faux négatifs)

La construction d'une fonction de coût convexe pour la classification

Fonction de coût logistique (log-loss)

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(h_{\theta}(x_i)) + (1 - y_i) \log(1 - h_{\theta}(x_i))]$$

On retrouve la même logique qu'en régression linéaire : minimiser $J(\theta)$ par descente de gradient, avec un gradient de forme similaire.

Pourquoi pas l'erreur quadratique ?

- Avec la sigmoïde, l'erreur quadratique devient non convexe : la descente de gradient risque de rester bloquée dans un minimum local
- La log-loss reste convexe pour un modèle logistique : elle garantit un minimum global unique
- Elle pénalise fortement une prédiction confiante et fautive (log tend vers l'infini)

Aller au-delà du binaire : one-vs-rest et régression softmax

One-vs-Rest (OvR)

Entraîne un classifieur logistique binaire par classe (« classe k contre le reste »), puis retient la classe dont le classifieur donne le score le plus élevé.

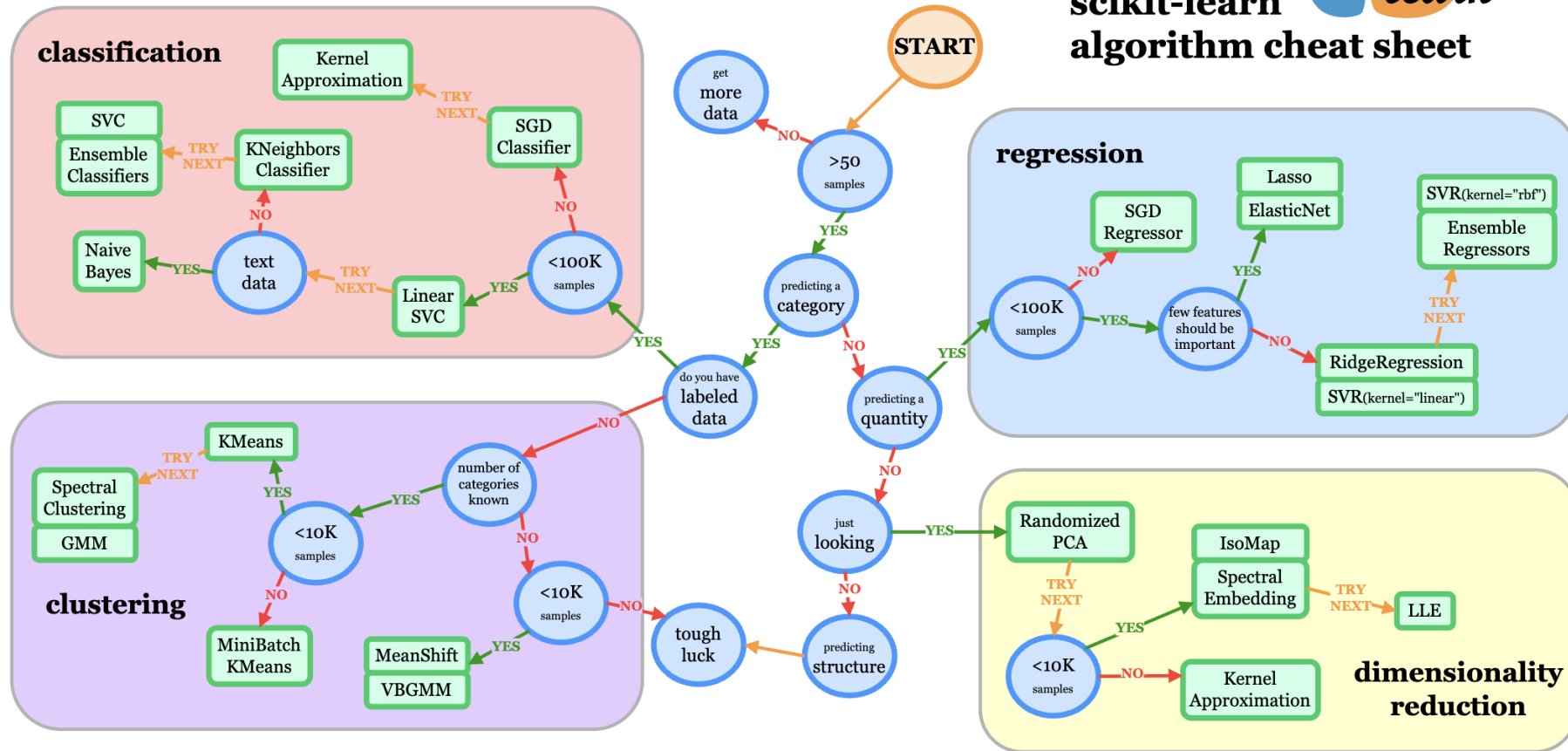
Régression softmax (multinomiale)

Généralise directement la sigmoïde à K classes : produit une distribution de probabilités sur toutes les classes qui somme à 1.

Pourquoi cette extension est utile

- Le problème OCR vu en début de journée (reconnaître un caractère parmi plusieurs) est un cas typique de classification multi-classes
- Scikit-learn le gère nativement : `LogisticRegression()` applique la softmax dès 3 classes (OvR : `OneVsRestClassifier`)

scikit-learn algorithm cheat sheet



05

La boîte à outils du Data Scientist

- Introduction aux outils
- Introduction à Python, Pandas et Scikit-learn
- Cas pratique n°1 : Prédire les survivants du Titanic

Introduction aux outils

Python	Langage de référence en Data Science : lisible, immense écosystème de bibliothèques scientifiques.
Jupyter Notebook	Environnement interactif mêlant code, résultats et texte — standard pour l'exploration de données.
Pandas	Bibliothèque de manipulation de données tabulaires (DataFrame) : chargement, nettoyage, agrégation.
NumPy	Calcul numérique performant sur tableaux et matrices — brique de base de l'écosystème Python data.
Scikit-learn	Bibliothèque de référence pour le machine learning « classique » : modèles, métriques, prétraitement.
Matplotlib / Seaborn	Bibliothèques de visualisation de données pour explorer et communiquer les résultats.

Introduction à Python, Pandas et Scikit-learn

Pandas — charger et explorer un dataset

```
import pandas as pd

df = pd.read_csv('data.csv')
df.head()
df.describe()
df['age'].fillna(df['age'].mean())
```

Scikit-learn — séparer les données

```
from sklearn.model_selection import (
    train_test_split)
from sklearn.linear_model import (
    LogisticRegression)

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.2)
```

Le workflow scikit-learn en 3 étapes

- `model = LogisticRegression()` → instancier le modèle
- `model.fit(X_train, y_train)` → entraîner sur les données d'entraînement
- `model.predict(X_test)` → prédire sur des données non vues

Le dataset Titanic en détail

Colonne	Type	Description
PassengerId	Identifiant	Identifiant unique du passager
Survived	Target (0/1)	1 si le passager a survécu, 0 sinon
Pclass	Ordinale (1/2/3)	Classe du billet — proxy du statut socio-économique
Sex, Age	Catégorielle / continue	Sexe et âge du passager (Age comporte des valeurs manquantes)
SibSp, Parch	Continues (entiers)	Nombre de frères/sœurs-époux, et de parents/enfants à bord
Fare, Embarked	Continue / catégorielle	Tarif payé et port d'embarquement (C, Q, S)

TP 1 — Prédire les survivants du Titanic — Exposé du problème

TP Mise en situation

Le dataset Titanic recense les passagers du paquebot (âge, sexe, classe, tarif, port d'embarquement...) ainsi qu'une variable indiquant s'ils ont survécu. Objectif : prédire la survie d'un passager à partir de ses caractéristiques.

1. Identifier la target : la variable « Survived » (0 = non, 1 = oui) → problème de classification binaire
2. Lister les features disponibles : Pclass, Sex, Age, SibSp, Parch, Fare, Embarked...
3. Formuler des hypothèses métier : le sexe, la classe ou l'âge influencent-ils la survie ?
4. Charger le dataset avec Pandas et observer sa structure (df.info(), df.describe())
5. Repérer les valeurs manquantes et les types de variables (numériques / catégorielles)

Première manipulation en Python

titanic_exploration.py

```
import pandas as pd

titanic = pd.read_csv('titanic.csv')

# Aperçu et structure du dataset
titanic.head()
titanic.info()
titanic.describe()

# Taux de survie global et par sexe
titanic['Survived'].mean()
titanic.groupby('Sex')['Survived'].mean()

# Valeurs manquantes par colonne
titanic.isnull().sum()
```

À l'issue de cette première manipulation, chaque participant a identifié les features candidates, la target et les premières pistes de nettoyage — base du travail de la journée 2.

Pipeline scikit-learn : enchaîner prétraitement et modèle

titanic_pipeline.py

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression

preprocess = ColumnTransformer([
    ('num', StandardScaler(), ['Age', 'Fare']),
    ('cat', OneHotEncoder(), ['Sex', 'Embarked']),
])

pipeline = Pipeline([
    ('prep', preprocess),
    ('model', LogisticRegression()),
])

pipeline.fit(X_train, y_train)
pipeline.predict(X_test)
```

Un pipeline garantit que les mêmes transformations sont appliquées à l'identique en entraînement et en prédiction — c'est un rempart efficace contre la fuite de données.

Jour 1 — Ce qu'il faut retenir

1

Poser le problème

Big Data, vocabulaire (feature/target), CRISP-DM : traduire une question métier en problème de classification ou de régression.

2

Comprendre les algorithmes

Régression linéaire et logistique : fonction hypothèse, fonction de coût convexe, descente de gradient et ses variantes.

3

Élargir la boîte à outils

k-NN et arbres de décision en complément, Python / Pandas / Scikit-learn et le pattern Pipeline.

4

Passer à la pratique

Premier modèle sur le dataset Titanic — base du jour 2 et de la première soumission Kaggle.

Rendez-vous demain pour transformer ce premier modèle en un modèle robuste : évaluation rigoureuse, nettoyage et enrichissement des données.

Q1

QCM — Jour 1

- 8 questions pour valider les fondamentaux de la journée
- La réponse s'affiche au clic, juste après chaque question

Parmi les propositions suivantes, laquelle NE fait PAS partie des « 5V » du Big Data ?

A Volume

B Vitesse

C Virtualisation

D Vérité

✓ **Bonne réponse : C.**
Les 5V couramment cités sont Volume, Vitesse, Variété, Vérité et Valeur. La virtualisation est une technologie d'infrastructure, pas l'une des dimensions caractérisant le Big Data.

Dans le cadre méthodologique CRISP-DM, que se passe-t-il si l'étape d'évaluation révèle une performance insuffisante ?

- ☐ A Le projet s'arrête immédiatement
- ☒ B On peut revenir à des étapes antérieures : préparation des données, voire compréhension métier
- ☐ C On change uniquement le nom du modèle
- ☐ D On ignore le résultat et on déploie quand même

☒ **Bonne réponse : B.**
CRISP-DM est un cycle itératif, pas un processus linéaire : une évaluation décevante renvoie fréquemment vers la préparation des données ou une reformulation du problème métier.

Dans un problème de Data Science, que désigne la « target » ?

A Une variable explicative utilisée en entrée du modèle

B La variable que l'on cherche à prédire

C Le sous-ensemble de données réservé au test

D Le taux d'erreur du modèle

 **Bonne réponse : B.**
La target (ou label) est la variable expliquée. Les autres colonnes utilisées pour la prédire sont les features (variables explicatives).

Quelle différence essentielle distingue l'apprentissage supervisé de l'apprentissage non supervisé ?

- A Le supervisé utilise Python, le non supervisé utilise R
- B Le supervisé dispose d'une target connue, le non supervisé n'en a pas
- C Le non supervisé est toujours plus précis
- D Le supervisé ne s'applique qu'aux images

Bonne réponse : B.

- ✓ En apprentissage supervisé, chaque observation d'entraînement est associée à une target connue. En non supervisé, l'algorithme explore la structure des données sans label fourni.

Pourquoi le test set ne doit-il jamais être utilisé pour choisir un modèle ou régler un hyperparamètre ?

- A Parce qu'il contient toujours moins de données que le training set
- B Parce que cela biaiserait l'estimation finale de la performance du modèle**
- C Parce que scikit-learn l'interdit techniquement
- D Parce qu'il ne contient que des valeurs manquantes

 **Bonne réponse : B.** Le test set doit rester une donnée totalement inédite pour le modèle et pour le processus de sélection, afin de donner une estimation non biaisée de la performance en conditions réelles.

Le problème « prédire le prix d'un bien immobilier » est un problème de :

- A** Classification
- B** Clustering
- C** Régression
- D** Détection d'anomalies

Bonne réponse : C.



La target (le prix) est une valeur numérique continue : il s'agit donc d'un problème de régression, à l'inverse d'une classification où la target serait une catégorie discrète.

Pourquoi la convexité de la fonction de coût est-elle recherchée en régression linéaire ?

- A Parce qu'elle rend le calcul plus rapide sur GPU
- B Parce qu'elle garantit l'existence d'un seul minimum global atteignable par descente de gradient
- C Parce qu'elle évite d'avoir à normaliser les données
- D Parce qu'elle supprime le besoin d'un jeu de test

Bonne réponse : B.



Une fonction convexe a la forme d'une cuvette unique : la descente de gradient est assurée de converger vers ce minimum global, sans risque de rester bloquée dans un minimum local.

Dans la formule $\theta \leftarrow \theta - \alpha \cdot \nabla J(\theta)$ de la descente de gradient, à quoi correspond α ?

A Le nombre d'observations du dataset

B Le taux d'apprentissage, qui contrôle la taille du pas à chaque mise à jour

C La probabilité prédite par le modèle

D Le nombre de features du dataset

Bonne réponse : B.



α (le learning rate) fixe l'amplitude du déplacement des paramètres à chaque itération : trop petit, la convergence est lente ; trop grand, l'algorithme peut osciller ou diverger.